Member-only story

Claude Code | Ai Coding | Software Engineering | Programming | LLM

Stop Using Claude Code Without Ponytail (It Beats Karpathy's Famous 4 Line CLAUDE.md)

A 100 line prompt got 157,000 GitHub stars, got debunked by a CTO with seven words, rebuilt its own benchmark, and then passed an independent JetBrains test. Here's what Ponytail actually does, what it really saves, and how it stacks up against the Karpathy rules I've been running myself.



Itai Spector Follow 12 min read · Oct 10, 2026

486 8 5

Ask your coding agent for a date picker and watch what happens.

It installs a library. It writes a wrapper component around that library. It adds a stylesheet. Then it opens a discussion about time zones that nobody asked for. In one real benchmark run on a real React codebase, that whole ritual came out to 404 lines of code.

The same agent, same task, same repo, with one plugin installed, wrote 23 lines. Most of them were this:

```
<!-- ponytail: browser has one -->
<input type="date">
```

That plugin is called **Ponytail**, and it has 157,500 stars on GitHub as I write this. It went from zero to that in under four months. I want to explain why it deserves a place in your Claude Code setup, why the number on its README is not the number you will get, and why I think it beats the other giant of this category: the famous "Karpathy" CLAUDE.md that sits at 217,000 stars and that I've been running myself.



Two rule sets, one agent. Only one of them came with receipts.

The guy with the ponytail

You know him. Every company has one. Long ponytail, oval glasses, has been at the company longer than the version control system. You show him the 50 lines you are proud of. He looks at them for a while, says nothing, and replaces them with one.

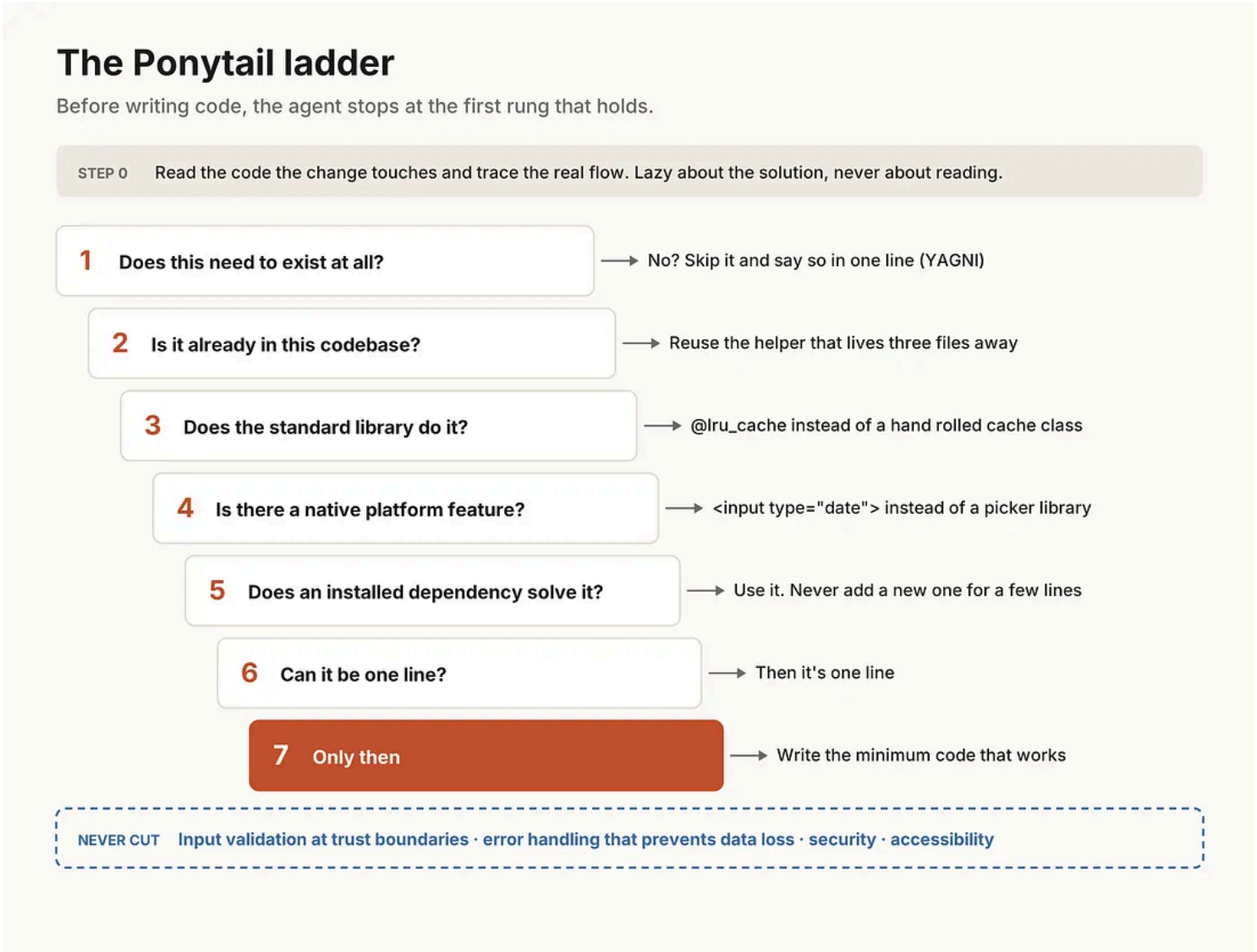
That's the whole pitch of Ponytail, built by developer Dietrich Gebert. It puts that guy inside your agent.



The project's own mascot. Source: DietrichGebert/ponytail on GitHub

Mechanically, it's a prompt. One `SKILL.md` file of roughly 100 lines, plus a lot of packaging to load that prompt into about 20 different agents: Claude Code,

Codex, Cursor, Copilot, Gemini CLI, OpenCode, Windsurf and more. The core idea is a decision ladder the agent climbs before it writes anything, stopping at the first rung that works.



The whole skill in one picture. Rungs paraphrased from Ponytail's SKILL.md.

The order is the clever part. "Already in this codebase" sits above the standard library, because the most common slop in AI generated code is reimplementing a helper that lives three files away. Native platform features sit above dependencies, which is why the date picker collapses into a browser input and a color picker collapses into `<input type="color">`.

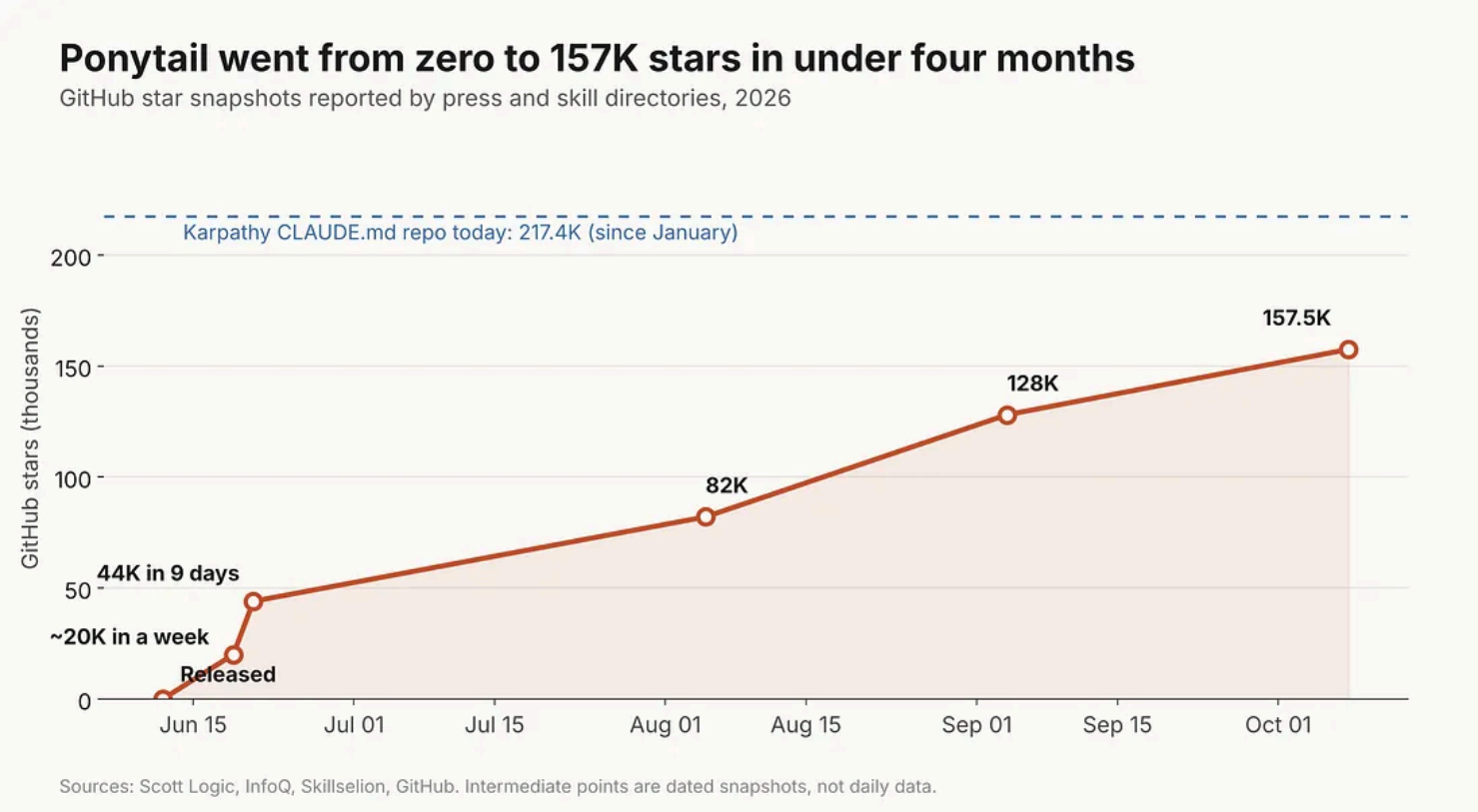
Two more rules keep it from being just "write less":

1. **The ladder runs after understanding, not instead of it.** The agent is told to read the code the change touches and trace the real flow before it picks a rung. Lazy about the solution, never about the reading. For bug fixes it's told to grep every caller and fix the shared function once, instead of patching only the path the ticket mentions.
2. **Some things are never on the chopping block.** Input validation at trust boundaries, error handling that prevents data loss, security, and accessibility basics are explicitly protected. When it does cut a real corner (a global lock, an $O(n^2)$ scan), it's supposed to leave a `ponytail:` comment naming the ceiling and the upgrade path.

On top of the default mode there are three intensity levels (lite, full, ultra) and a few commands I find more interesting than the core mode. `/ponytail-review` reads your current diff and hands back a delete list. `/ponytail-audit` does the same for the whole repo. `/ponytail-debt` collects every `ponytail:` shortcut into a ledger so "later" doesn't quietly become "never."

157K stars for a markdown file

Here's how fast this thing moved, based on the snapshots I could find in coverage and listings.

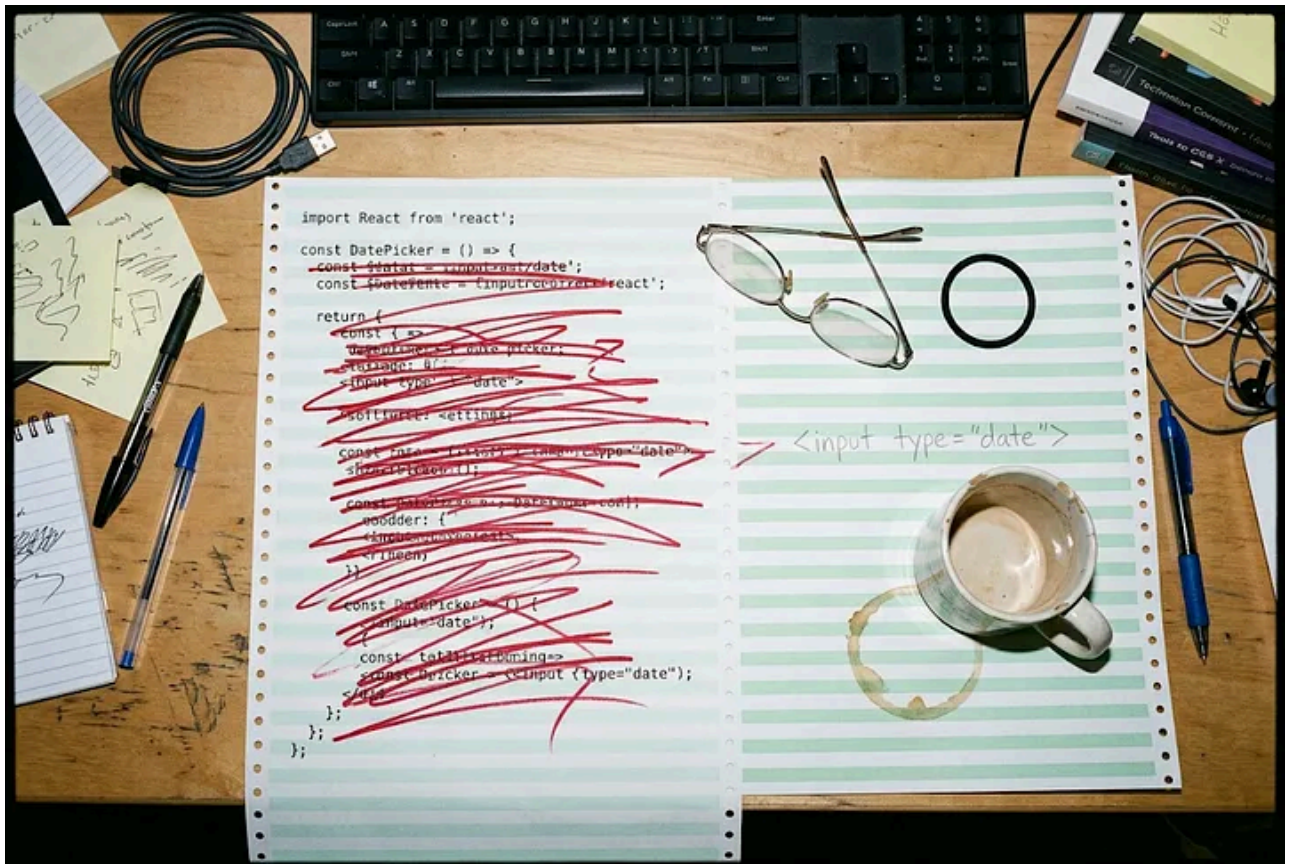


Ponytail was released on June 12, 2026. Colin Eberhardt, CTO at Scott Logic, counted around 20,000 stars after roughly a week. InfoQ reported it passed 44,000 in nine days and 82,000 by early August. Today it's at 157,500 with 8,500 forks.

You'd expect the backlash to follow, and it did. Colin went through the repo and found 6,232 lines of code across 90 files wrapped around a 100 line prompt that, in his reading, restates YAGNI, a principle from the 1990s. On Hacker News someone asked whether this was the new leftpad.

Honestly? They have a point about the packaging. They have a point about YAGNI too. Nothing in Ponytail is a new idea. Every senior developer already knows "check if the browser does it before you install a library."

But that's exactly the thing. Your senior developers know it. Your agent, left alone, doesn't act on it. Andrej Karpathy wrote in January that models will happily build a bloated 1,000 line construction and it's up to you to ask "umm couldn't you just do this instead?" before they cut it to 100. Ponytail is that question, asked automatically, before the 1,000 lines exist.



What Ponytail is supposed to do to your diff, before the diff exists.

The debunking is the best part of this story

This is where Ponytail stops being just another viral repo for me.

The original README claimed 80% to 94% less code. Colin ran the benchmark himself and found the problem. The tests were tiny, single prompt tasks like "write an email validator," and the baseline model had no coding agent system prompt. So it answered like a chatbot: five options, usage examples, commentary. The benchmark counted all of that as "lines of code."

He tightened the baseline step by step on Haiku:

- Plain baseline: 108 lines on average
- Add "Provide just one example": 16 lines
- Add "Follow YAGNI principles": 10.4 lines
- Ponytail: 8.25 lines
- Add "and one-liner solutions": 6.9 lines

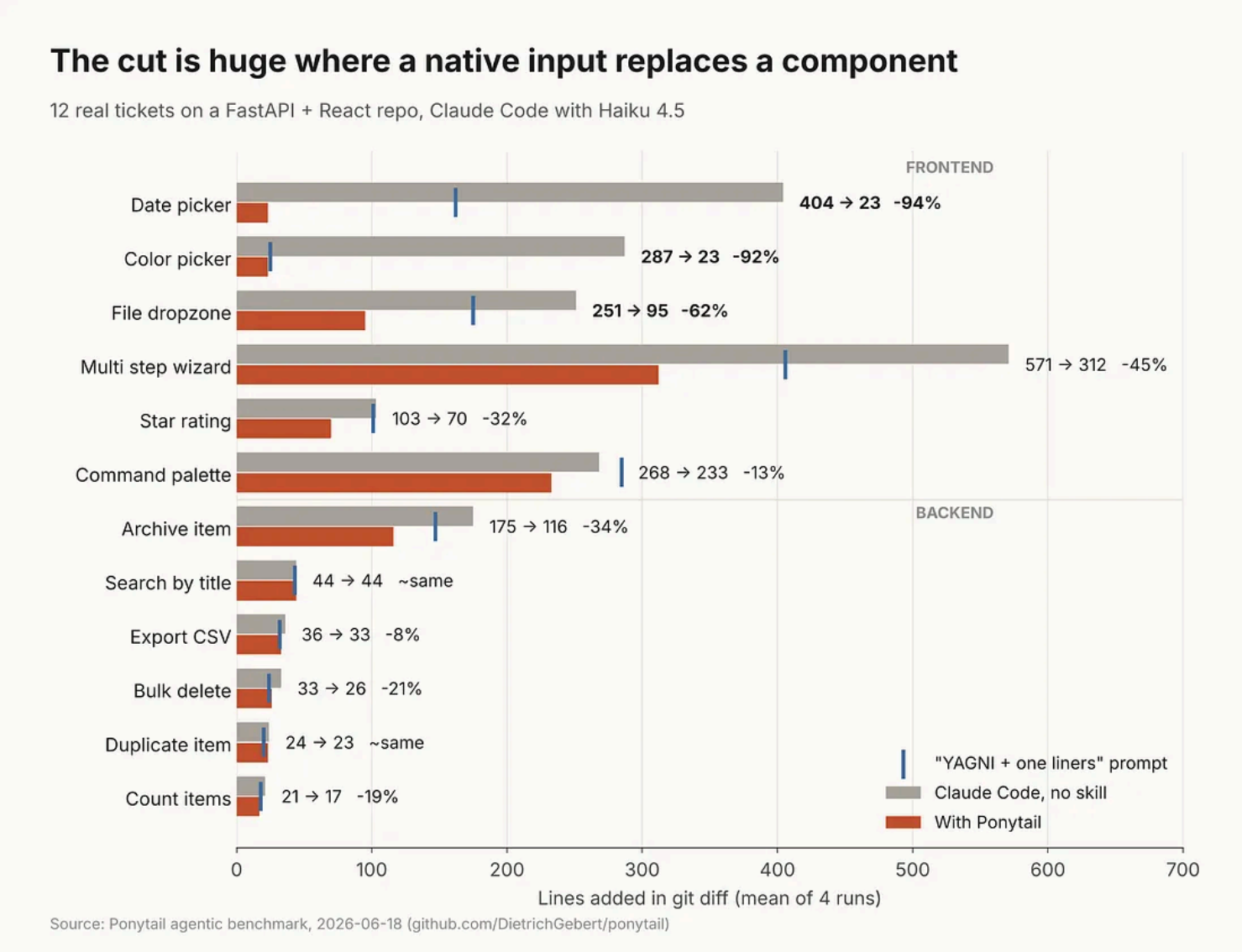
Seven words beat Ponytail on its own benchmark. Brutal.

Most viral AI projects respond to that by going quiet or getting defensive. Gebert did the opposite. He agreed with the critique in issue #126, rebuilt the benchmark from scratch, and published a smaller number.

The new setup is the right one. Real headless Claude Code sessions, not a bare API call. A real public repo (tiangolo's full stack FastAPI and React template). Twelve feature tickets, four runs each, and lines counted from the `git diff` the agent leaves behind. The baseline is the same agent with no skill, so chattiness can't inflate anything. He even included Colin's seven word prompt as its own arm.

And while doing it, he found a bug in his own harness that made the baseline secretly run Ponytail too. Ponytail installs a session start hook, and it was firing on every arm. He fixed it and wrote it up. That's the kind of detail you only publish if you actually want the numbers to be right.

The result: 54% less code on average, 22% fewer tokens, 20% lower cost, 27% faster. The average hides the real story, though:



The blue tick is Colin Eberhardt's seven word prompt. Brilliant on one task, worse than nothing on another.

The huge cuts live exactly where a native platform feature can replace a custom build. Date picker, 404 to 23. Color picker, 287 to 23. File dropzone, 251 to 95. On backend CRUD endpoints that were already minimal, everything converges and Ponytail barely moves the needle. It doesn't invent savings, and it never wrote more than the baseline.

Colin's seven word prompt turned out to be the interesting control. It was brilliant on the color picker (25 lines) and useless on the date picker (162). On the command palette it wrote more than the baseline. And in a separate

safety tier, where the generated code ran against adversarial input like path traversal and SQL injection, the seven word prompt was the only arm that dropped a guard, once in 20 runs. It wrote the fewest lines on the path traversal task, and the lines it skipped were the check. Ponytail kept them and stayed 100% safe.

That's the difference between "write less" and "be lazy, not negligent." One is a vibe. The other is a rule with exceptions written down.

Then someone who had no stake in it ran the numbers

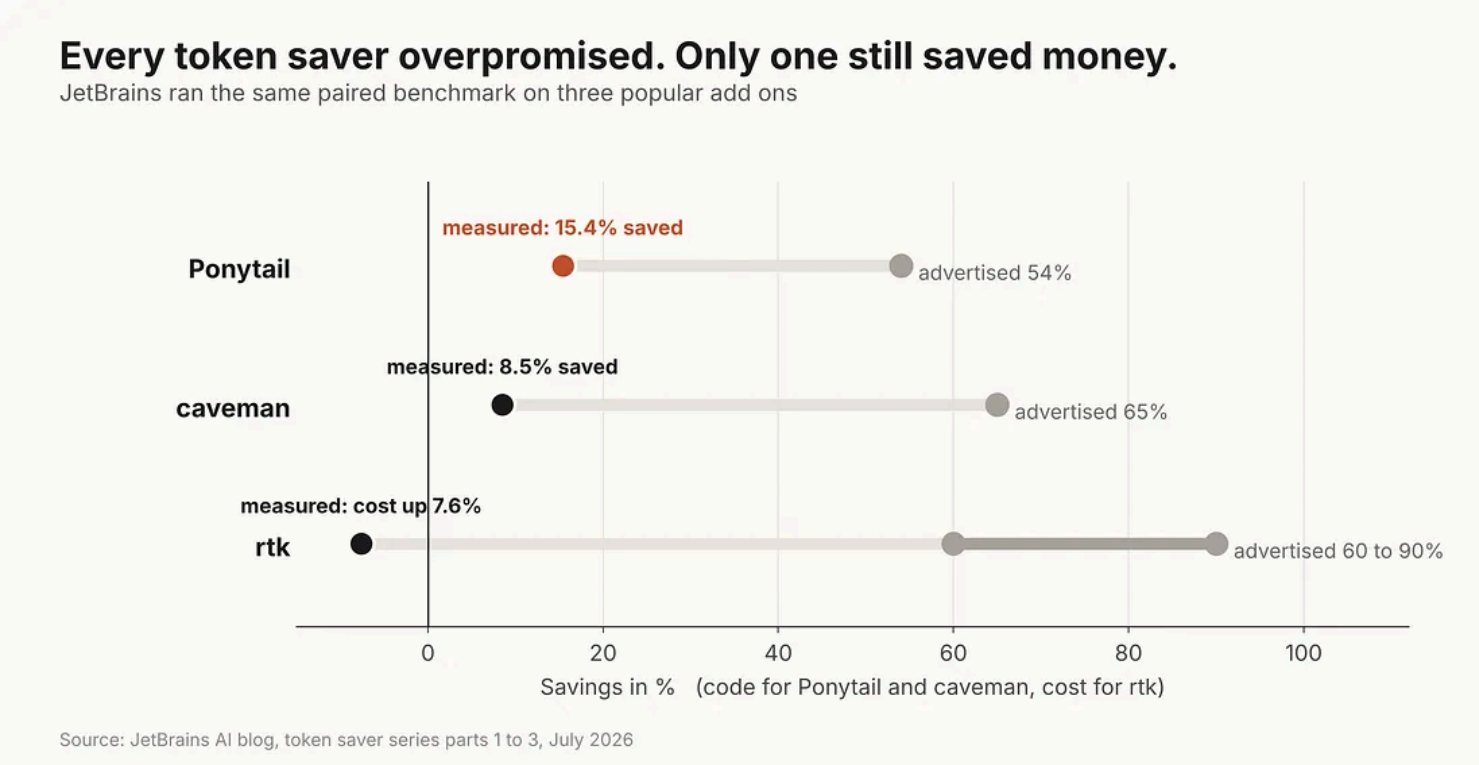
A benchmark the author runs himself, even an honest one, is still the author picking the tasks. What convinced me was the JetBrains test.

JetBrains has been running a series on "token saver" add ons for coding agents, using the same paired A/B setup each time. For Ponytail they ran 80 paired tasks from SkillsBench, a benchmark nobody built with Ponytail in mind, using Claude Code with Claude Sonnet 5. They spent about $246 on agent runs and audited every trial to make sure the ruleset actually reached the model in one arm and not the other.

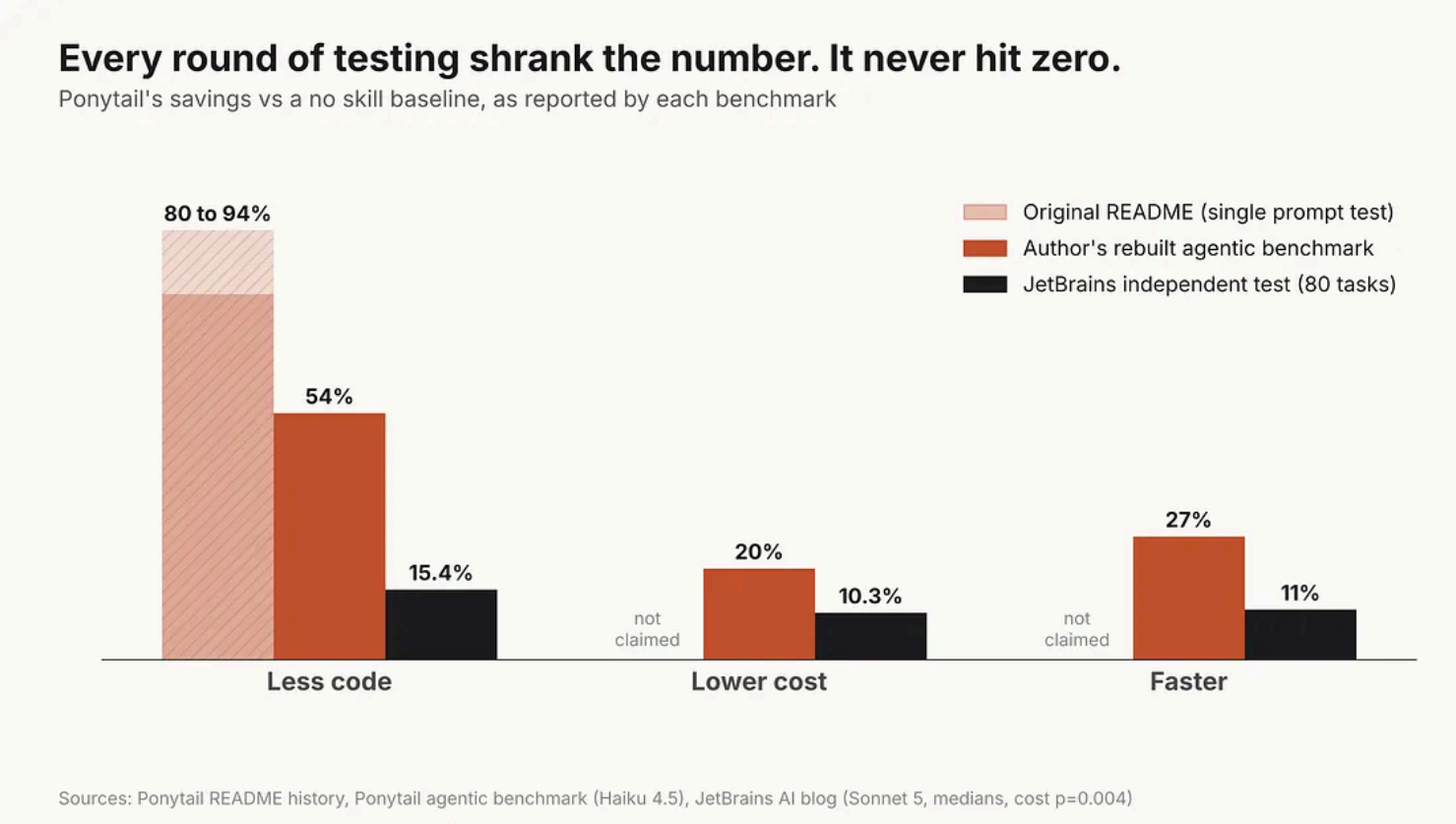
What they measured:

- About 15% less code (10,205 lines became 8,756)
- 10.3% lower cost, with p=0.004, which is the solid result
- About 11% faster
- No quality difference they could detect: 65 tasks scored identically, 9 slightly worse, 6 slightly better

So roughly a quarter to a half of the advertised effect. Before you shrug at that, look at the rest of the series:



Caveman, the terse prose skill that passed 70,000 stars back in June, advertised 65% savings and measured 8.5% less code. rtk advertised 60% to 90% and actually made things 7.6% more expensive. Ponytail is the first tool in the series that saved money with a statistically solid signal. Same pattern as Gebert's own data, too: on bigger builds the cut reached 31%, and on tasks that were already lean it was basically zero. The more your agent overbuilds, the more Ponytail has to cut.



Different models, different tasks, same direction. The JetBrains figures are medians, the author's are means.

Two findings from JetBrains matter more for you than the headline, though.

If you install it as a plain skill, it does nothing. JetBrains dropped the `SKILL.md` into a skills folder and let Claude Code decide when to use it. Across ten sessions, it activated itself zero times. Not rarely. Zero. The only setup that produced results is the plugin, which uses a session start hook to inject the ruleset whether the model wants it or not. So don't copy the file. Install the plugin:

```
/plugin marketplace add DietrichGebert/ponytail
/plugin install ponytail@ponytail
```

The model follows the ladder but skips the paperwork. Across 80 trials, the `ponytail:` shortcut comment showed up exactly once. So if you were counting on `/ponytail-debt` to track every corner it cut, don't. The ladder works. The bookkeeping mostly doesn't.

And then there's the Karpathy file

The other repo everyone mentions in the same breath is andrej-karpathy-skills. 217,000 stars, almost 22,000 forks. People call it "Karpathy's 4 lines."

Two corrections, because they matter. Karpathy didn't write it. Developer Forrest Chang did, turning Karpathy's January 26 post about weeks of heavy Claude coding into a CLAUDE.md. Karpathy diagnosed the problems; he didn't publish the fix. And it isn't 4 lines. It's 4 principles in a file of about 65 lines:

1. **Think Before Coding.** State assumptions, surface tradeoffs, ask when unclear.
2. **Simplicity First.** No unrequested features, abstractions or configurability.
3. **Surgical Changes.** Touch only what you must, match the existing style, don't refactor what isn't broken.
4. **Goal-Driven Execution.** Turn tasks into verifiable goals, like "write a test that reproduces the bug, then make it pass."

It's a good file. I'd hand it to a junior developer. And I've been running it in one of my own projects for a while, so here's my honest report.

I can't give you a benchmark. I didn't measure anything. My gut says the rules have an effect, but I'm never sure how much of that is the file and how much is that current models are already pretty good on their own. If I'm honest, it's 50/50.

My real friction is Surgical Changes. I ask for a surgical change, and I get exactly that: a precise, minimal edit to the spot I pointed at. Which is sometimes not the full solution I was aiming for. It's surgical, and it's half baked. The model touched only what it had to, and the thing still isn't done, so I'm back for a second and third round to finish what one round should have covered.



Surgical: the patch is perfect. Done: not quite.

Interestingly, Ponytail has a rule aimed right at that gap. Its bug fix rule says the smallest change in the wrong place isn't lazy, it's a second bug. It tells the agent to find every caller and fix the root cause in the shared function, rather than patching the one path the ticket names. Karpathy's file optimizes for a small diff. Ponytail optimizes for a small diff that is actually complete. I haven't run Ponytail yet, so I can't tell you it solves my half baked problem in practice. But on paper, it's the first rule set I've seen that treats "too surgical" as a failure mode.

Ponytail vs Karpathy, side by side

Ponytail vs the Karpathy CLAUDE.md

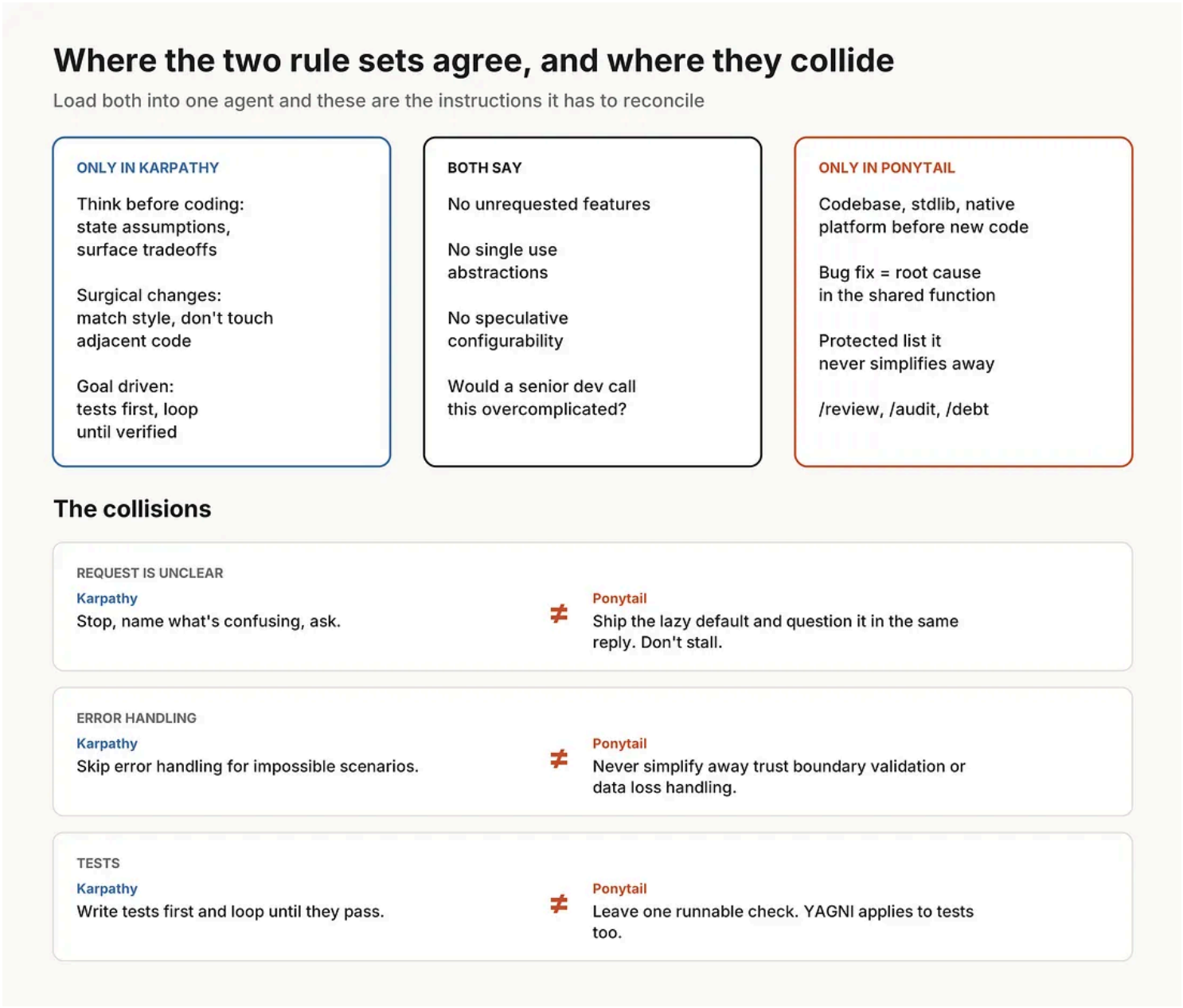
Two of the most starred rule sets for coding agents, compared on what actually matters

	Ponytail	Karpathy CLAUDE.md
Written by	Dietrich Gebert	Forrest Chang, based on Karpathy's Jan 26 post
GitHub stars / forks	157.5K / 8.5K	217.4K / 21.9K
The actual rules	One ~100 line SKILL.md	One ~65 line CLAUDE.md, 4 principles
Core question	What should be built?	How should the agent behave?
When the request is unclear	Ship the lazy default, question it in the same reply	Stop, name the confusion, ask
Error handling	Never cut validation, security, data loss, a11y	None for impossible scenarios
Bug fixes	Find every caller, fix the root cause once	Reproduce with a test, then make it pass
How it reaches the model	Plugin hook (as a plain skill: 0 of 10 activations)	Always in context as CLAUDE.md
Agents supported	About 20	Claude Code and Cursor
Extras	/review, /audit, /debt commands, 3 intensity levels	None
Own benchmark	Yes, rebuilt after public critique	None
Independent test	JetBrains: -15% code, -10.3% cost	None found

Sources: GitHub repos as of Oct 8, 2026; Ponytail SKILL.md; andrej-karpathy-skills CLAUDE.md; JetBrains AI blog, July 2026

The star counts suggest these are competing products. They're really different kinds of files. The Karpathy file is about how the agent should **behave**: ask, stay in your lane, verify. Ponytail is about what the agent should **build**: check the codebase, the standard library and the platform first, and write the minimum.

They overlap on one principle, Simplicity First. They also collide in two places, and this is the part no star count will tell you.



If you run both, decide which side wins each of these before your agent decides for you.

Ask vs ship. Karpathy's file tells the agent that if something is unclear, it should stop and ask. Ponytail says the opposite for complex requests: ship the lazy version and question the requirement in the same response, and "Never stall on an answer you can default." Load both and you're giving the model two instructions for the same moment. Which one wins depends on the model and the phrasing, which means you no longer know what you'll get.

What counts as necessary error handling. Karpathy's file says no error handling for impossible scenarios. Fair enough, but "impossible" is a judgment call the model makes. Ponytail draws a harder line: validation at trust boundaries, data loss handling, security and accessibility never get simplified away. That's the line that kept the path traversal check alive in Gebert's safety tier.

There's also a quieter difference in how each one actually reaches the model. A CLAUDE.md is always in context, so the Karpathy rules don't have an activation problem. Ponytail, as JetBrains showed, only works through its hook. If you install Ponytail the lazy way (ironically), you get nothing.

So should you install it?

I haven't installed Ponytail on a real project yet, so I won't pretend to give you a verdict from experience. Here's where the evidence leaves me, and what I'm doing next.

Yes, install it, as the plugin. It's one of the very few prompt packages in this category that published a benchmark, accepted a public takedown of it, rebuilt it properly, and then survived an independent test by a company with no stake in it. Expect about 10% off your bill and around 15% less code on a typical workload, more if your agent likes to overbuild. That's not the README number. It's still a real number, which is rarer than it should be.

Use `/ponytail-review` even if you hate rules that are always on. A delete list for your current diff costs nothing and is the feature your code reviewers will like most.

Don't stack it blindly on top of the Karpathy file. If you already run the

Open in app ↗

Medium Search Play Games Write

Ponytail, or remove the conflicting lines. Two rule sets pulling in different directions is worse than either one alone.

Don't expect magic on code that's already lean. Backend CRUD, data scripts and small fixes won't shrink much, and that's fine. The value is in the date picker moments, where your agent was about to install a library for something the browser ships for free.

Colin Eberhardt's bigger point still stands after all of this: skills are multiplying with no evaluation standard behind them, and most of the ones with six figure star counts have never proven anything. The Karpathy file, for all its stars, has no benchmark at all. Ponytail does, because someone forced it to.

That's the real reason not to sleep on it. Not the 157,000 stars. The receipts.

Claude Code + | Ai Coding + | Software Engineering + | Programming ✓

LLM +



Written by Itai Spector

94 followers · 10 following

Senior frontend dev & musician. Hands-on guides to Claude Code, MCP and the AI coding tools that launch every week.

Follow

Responses (8)

Alex Mylnikov

What are your thoughts?

Dave Kurian
Oct 10

The JetBrains finding that the plain skill activated itself zero times across ten sessions is the most practical detail here, since copying the SKILL.md is exactly what most people would do. I also liked that you separated the two kinds of files... more

8 2 replies Reply

Kamrun Nahar she/her
Oct 11

ah, a masterclass in nuance

5 Reply

Faras Samith him
Oct 11

Amazing

4 Reply

See all responses

More from Itai Spector

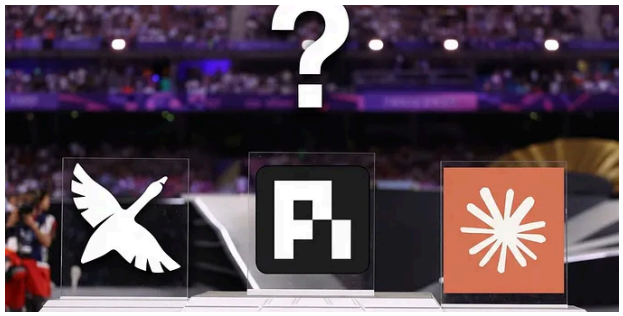


Itai Spector · Sep 24

How to Connect Jev to Claude in 5 Minutes (and What It's Actually...

Jev is TypeSafe's new model that answers with probabilities instead of words. It can't replac...

148 3

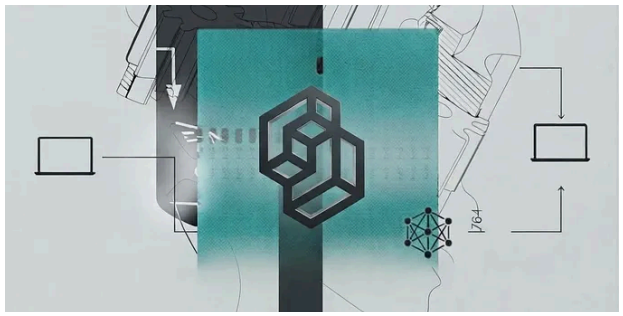


Itai Spector · Oct 4

The "Claude Code Killer" Hype, Six Months Later: What Actually...

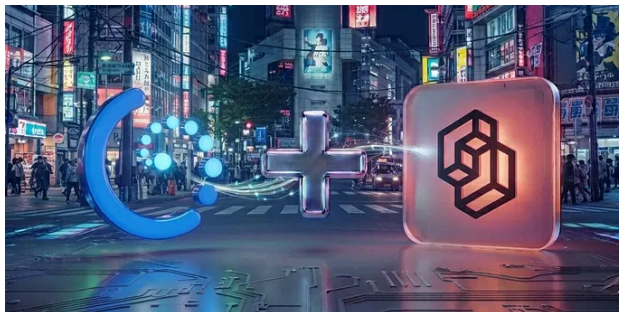
Anthropic closed the pricing loophole that made Goose worth switching for. Pi's...

49 1 1



Itai Spector · Sep 30

Jev Router for Claude Code Saves Less Than You Think. Here's What...



Itai Spector · Oct 13

Forget Jev. Meet Laya, the Free, Transparent Decision Model That...

Everyone is installing a Jev router to cut their Claude Code bill. The one project that...

62 1 1

Laya does what TypeSafe's Jev does: typed decisions with calibrated probabilities, no te...

54 2

See all from Itai Spector